

Programmation procédurale : les fichiers



On parle de quoi ?

1. [Le concept de fichier](#)
2. [Structures et fichiers](#)
3. [Deux modes de support](#)
4. [Opérations sur un fichier](#)
5. [Conseil pour un code propre](#)

Le concept de fichier

Un **fichier** est un ensemble de données qui concernent un même thème.

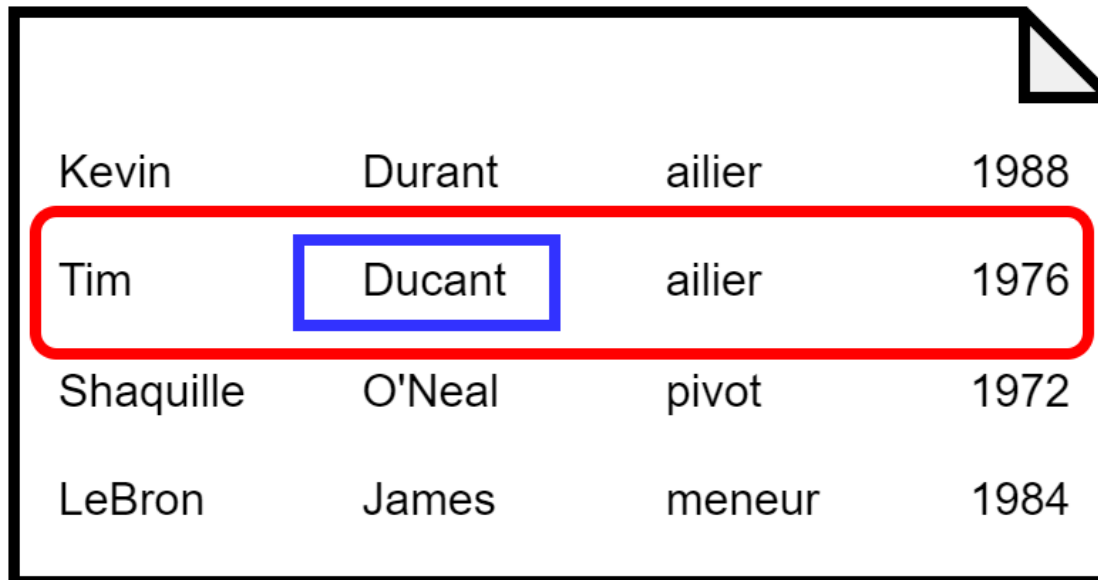
→ par exemple, un fichier contenant les joueurs d'une équipe de Basket

Un fichier est composé d'**enregistrements**.

→ par exemple, un enregistrement correspond à un joueur d'une équipe de Basket

Un enregistrement est composé de **champs**.

→ par exemple, un champ qui correspond au nom du joueur



Kevin	Durant	ailier	1988
Tim	Ducant	ailier	1976
Shaquille	O'Neal	pivot	1972
LeBron	James	meneur	1984

Le concept de fichier

Un champ possède 3 caractéristiques :

- un nom (ex. "Poste")
- un type (ex. entier)
- une longueur (fixe ou variable)

Longueur fixe

Kevin	Durant	ailier	1988
Tim	Ducant	ailier	1976
Shaquille	O'Neal	pivot	1972
LeBron	James	meneur	1984

Longueur variable

```
Kevin#Durant#ailier#1988  
Tim#Ducant#ailier#1976  
Shaquille#O'Neal#pivot#1972  
LeBron#James#meneur#1984
```

Structures et fichiers

Un enregistrement en C est construit à l'aide d'une structure.

Si on reprend l'exemple avec l'équipe de Basket :

```
#define TAILLE_NOM_MAX 150
#define TAILLE_PRENOM_MAX 150
#define TAILLE_POSTE_MAX 50

struct joueur{
    char nom[TAILLE_NOM_MAX];
    char prenom[TAILLE_PRENOM_MAX];
    char poste[TAILLE_POSTE_MAX];
    int dateNaissance;
};

typedef struct joueur Joueur;
```

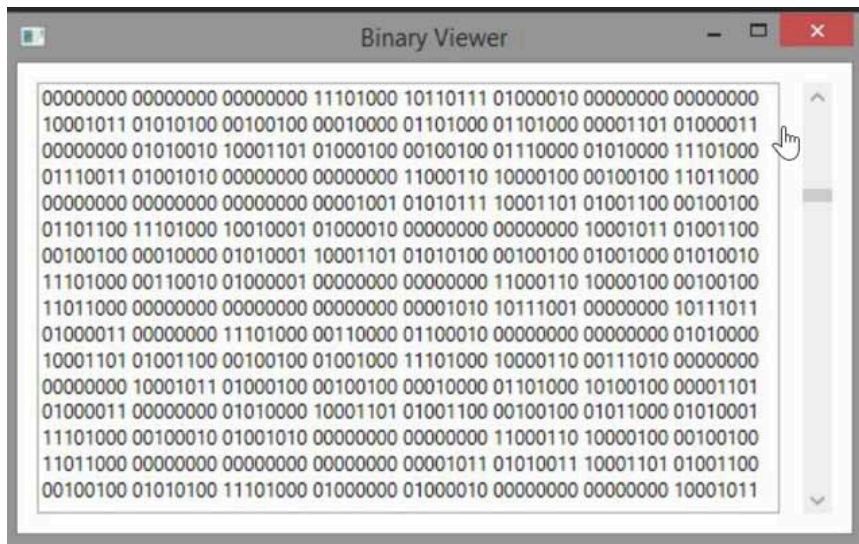
Deux modes de support

Fichier texte

- ✓ Lisible et compréhensible à l'oeil nu dans un éditeur de texte
- ✗ Portabilité non assurée car le codage du passage à la ligne dépend de l'OS

Fichier binaire

- ✓ Portabilité assurée (tous les ordinateurs du monde savent lire le binaire)
- ✗ Illisible à l'oeil nu



Opérations sur un fichier

L'ouverture

L'**ouverture** est l'opération indispensable avant tout travail sur le fichier.

Elle permet au programme d'avoir accès au fichier en stockant les données dans le buffer.

L'ouverture se fait avec la fonction `fopen_s` :

```
FILE *pFichier;  
fopen_s(&pFichier, "C:\\Lakers.dat", "rb");  
if(pFichier == NULL) printf("Fichier introuvable !\n");
```

→ `pFichier` est un **pointeur** : il s'agit d'une variable qui contient l'adresse d'une autre variable

→ ici, `pFichier` est un pointeur de type `FILE` : il pointe donc vers une zone dans un fichier

Opérations sur un fichier

L'ouverture

Accès fichier texte/binaire	Type d'ouverture	Fichier existant	Fichier inexistant
w / wb	Ecriture	Contenu écrasé	Création
r / rb	Lecture	Positionnement au début par défaut	Pointeur = NULL (erreur)
a / ab	Ecriture	Positionnement à la fin par défaut	Création
w+ / wb+	Lecture/Ecriture	Contenu écrasé	Création
r+ / rb+	Lecture/Ecriture	Positionnement au début par défaut	Pointeur = NULL (erreur)
a+ /ab+	Lecture/Ecriture	Positionnement à la fin par défaut	Création

Opérations sur un fichier

La lecture

La **lecture** permet d'obtenir des informations se trouvant dans le fichier.

- soit l'enregistrement complet
- soit certains champs de certains enregistrements
- soit plusieurs enregistrement

Opérations sur un fichier

La lecture

La lecture se fait avec la fonction `fread_s` :

```
FILE *pFichier;
Joueur joueur;
fopen_s(&pFichier, "C:\\Lakers.dat", "rb");
if (pFichier == NULL) {
    printf("Fichier introuvable !\n");
}
else {
    fread_s(&joueur, sizeof(joueur), sizeof(joueur), 1, pFichier);
    // joueur contient les données du premier joueur dans le fichier
    // pFichier indique la position qui suit le dernier octet lu
    // (dans ce cas-ci, le joueur suivant)
}
```

Opérations sur un fichier

La lecture

Comment savoir quand arrêter la lecture lorsqu'il n'y a plus de données à lire ?

- Première solution : utiliser `feof`

```
// Toujours une lecture avant d'utiliser feof !!
fread_s(&joueur, sizeof(joueur), sizeof(joueur), 1, pFichier);
while(!feof(pFichier)){
    fread_s(&joueur, sizeof(joueur), sizeof(joueur), 1, pFichier);
}
```

- Deuxième solution : utiliser la valeur de retour de `fread_s`

```
while(fread_s(&joueur, sizeof(joueur), sizeof(joueur), 1, pFichier) != 0){
    // affichage
}
```

Opérations sur un fichier

L'écriture

L'**écriture** consiste à écrire des informations dans le fichier (nouveaux champs, nouveaux enregistrements).

Opérations sur un fichier

L'écriture

L'écriture se fait avec la fonction `fwrite` :

```
FILE *pFichier;
Joueur joueur;
fopen_s(&pFichier, "C:\\Lakers.dat", "ab"); // pourquoi ab ?
if (pFichier == NULL) {
    printf("Fichier introuvable !\n");
}
else {

    strcpy_s(joueur.nom, TAILLE_NOM_MAX, "Jordan");
    strcpy_s(joueur.prenom, TAILLE_PRENOM_MAX, "Michael");
    strcpy_s(joueur.poste, TAILLE_POSTE_MAX, "arriere");
    joueur.dateNaissance = 1963;

    fwrite(&joueur, sizeof(joueur), 1, pFichier);

    //pFichier indique la position qui suit le dernier octet écrit
    // (dans ce cas-ci, après le nouveau joueur ajouté)
}
```

Opérations sur un fichier

L'écriture

L'écriture peut également se faire avec `fprintf` avec les fichiers texte(déconseillé).

→ généralement utilisé pour tester son code

```
FILE *pFichier;
Joueur joueur;
fopen_s(&pFichier, "C:\\Lakers.dat","a"); // pourquoi ab ?
if (pFichier == NULL) {
    printf("Fichier introuvable !\n");
}
else {

    strcpy_s(joueur.nom, TAILLE_NOM_MAX, "Jordan");
    strcpy_s(joueur.prenom, TAILLE_PRENOM_MAX, "Michael");
    strcpy_s(joueur.poste, TAILLE_POSTE_MAX, "arriere");
    joueur.dateNaissance = 1963;

    fprintf(pFichier, "%s %s %s %d",
    joueur.nom, joueur.prenom, joueur.poste, joueur.dateNaissance);
}
```

Opérations sur un fichier

La fermeture

La fermeture coupe le lien entre le programme et le fichier.

La fermeture se fait avec la fonction `fclose` :

```
FILE *pFichier;  
Joueur joueur;  
fopen_s(&pFichier, "C:\\Lakers.dat","rb");  
if (pFichier == NULL) {  
    printf("Fichier introuvable !\n");  
}  
else {  
    // lecture(s), écriture(s),...  
    fclose(pFichier);  
}
```

→ **Indispensable avant la terminaison du programme !**

Se positionner dans le fichier

Parfois, il est nécessaire de déplacer manuellement la position du pointeur.

Déplacement à partir du début du fichier : **SEEK_SET**

```
fseek(pFichier, N, SEEK_SET);
```

Déplacement à partir de la position courante : **SEEK_CUR**

```
fseek(pFichier, N, SEEK_CUR);
```

Déplacement à partir de la fin du fichier : **SEEK_END**

```
fseek(pFichier, N, SEEK_END);
```


Conseil pour un code propre

```
void main(void){
    FILE *pFichier;
    Joueur joueur;
    fopen_s(&pFichier, "C:\\Lakers.dat", "rb");
    if (pFichier == NULL) {
        printf("Fichier introuvable !\n");
    }
    else {
        // lecture(s), écriture(s),...
        fclose(pFichier);
        fopen_s(&pFichier, "C:\\Lakers.dat", "ab");
        // lecture(s), écriture(s),...
        fclose(pFichier);
        fopen_s(&pFichier, "C:\\Lakers.dat", "wb");
        // lecture(s), écriture(s),...
        fclose(pFichier);
    }
}
```

→ Comment améliorer ce code ?

Conseil pour un code propre

```
#define CHEMIN_FICHIER "C:\\Lakers.dat"
void main(void){
    FILE *pFichier;
    Joueur joueur;
    fopen_s(&pFichier, CHEMIN_FICHIER,"rb");
    if (pFichier == NULL) {
        printf("Fichier introuvable !\n");
    }
    else {
        // lecture(s), écriture(s),...
        fclose(pFichier);
        fopen_s(&pFichier, CHEMIN_FICHIER,"ab");
        // lecture(s), écriture(s),...
        fclose(pFichier);
        fopen_s(&pFichier, CHEMIN_FICHIER,"wb");
        // lecture(s), écriture(s),...
        fclose(pFichier);
    }
}
}
```